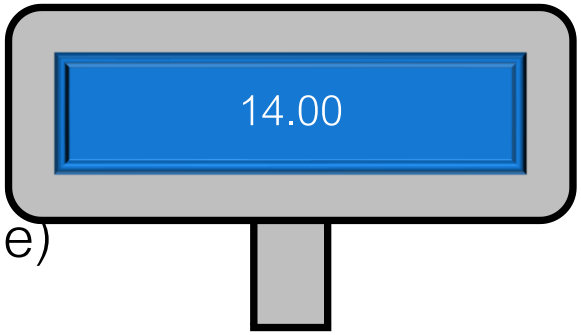
GREEDY ALGORITHM

ปราการ อุณจักร
ภาควิชาวิทยาการคอมพิวเตอร์, คณะวิทยาศาสตร์, มหาวิทยาลัยเชียงใหม่
เนื้อหาบางส่วนจาก ผศ.ดร.จักริน ชวชาติ

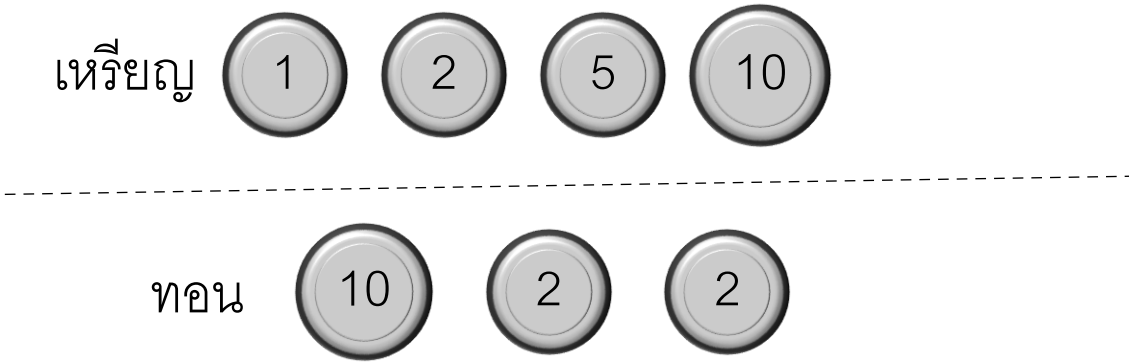
Optimization Problem

- ปัญหาส่วนมาก (โดยเฉพาะในการแข่ง) จะเป็นปัญหา Optimization
  - หาคำตอบ (Solution) ที่ดีที่สุด (Optimal Solution) ที่เป็นไปตามเงื่อนไขที่กำหนด
  - "ดีที่สุด" – ขึ้นอยู่กับปัญหา
    - หาเส้นทางที่สั้นที่สุด / ใช้เวลาน้อยที่สุด จากจุด A ไปจุด B
    - จัดของใส่กระเป๋าให้ได้มูลค่ามากที่สุด โดยที่น้ำหนักรวมไม่เกินที่กำหนด
    - จัดตารางเรียนที่ใช้ห้องเรียนน้อยที่สุด ให้ครบทุกวิชา

Example: Making Change



- ตัวอย่างปัญหา: ทอนเงิน (Making Change)
  - จะต้องทอนเงิน *n* บาท
  - มีเหรียญอยู่ *k* ประเภท เหรียญแต่ละประเภทมีค่า $c_1, c_2, \ldots, c_k$ เรียงจากค่าน้อยไปค่ามาก
  - จงหา**<u>จำนวนเหรียญที่น้อยที่สุด</u>**ที่สามารถทอนเงิน *n* บาทได้



Greedy Algorithm

- ในการแก้ปัญหา จะมีการเลือกตัวเลือกเป็นขั้น ๆ
- Greedy Algorithm **เลือก** คำตอบ(ทางเลือก/ตัวเลือก)ที่เหมาะสมที่สุดในแต่ละขั้น โดยหวังว่าโดยรวมหรือสุดท้ายแล้วจะพบคำตอบที่เหมาะสมที่สุด
- พูดง่ายๆ คือเป็นปัญหาที่สามารถถูกแก้ได้ด้วยการเลือกตัวเลือก/ทางเลือกที่ดูว่าดีที่สุด ณ จังหวะนั้น ในทุก ๆ ขั้นตอน
- ไม่พิจารณาทางเลือกก่อนหน้าอีกครั้ง เลือกแล้วเลือกเลยเนื่องจาก <u>จะตัดสินใจโดยอิงจากข้อมูลปัจจุบันโดยไม่มองไปข้างหน้า</u>

Greedy Algorithm (cont.)

- เป็นวิธีที่ง่าย และมีประสิทธิภาพ **<u>แต่อาจไม่ได้คำตอบที่ดีที่สุด</u>**
- โดยทั่วไป Greedy algorithm นี้จะทำงานได้ดีที่สุดเมื่อปัญหามีคุณสมบัติต่อไปนี้:
  - Greedy choice property: คำตอบที่ดีที่สุดสามารถสร้างได้โดยการเลือกทางเลือกที่ดีที่สุดในแต่ละขั้นตอน
  - Optimal substructure: คำตอบที่ดีที่สุดของปัญหาประกอบด้วยคำตอบที่ดีที่สุดของปัญหาส่วนย่อยของมัน

โครงของ Greedy algorithm

- Greedy algorithm จะแก้ปัญหาโดยในแต่ละขั้นจะเลือกทางเลือกที่ดีที่สุดโดยหวังว่าจะได้คำตอบโดยรวมสุดท้ายที่ดีที่สุด
- วิธีการทำงานโดยทั่วไปมีดังนี้:
  1. เริ่มต้นด้วยสถานะเริ่มต้นของปัญหา (จุดเริ่มต้นที่เริ่มตัดสินใจเลือก)
  2. พิจารณาทางเลือกทั้งหมดที่มีหรือที่เลือกได้ขณะนั้น
  3. เลือกทางเลือกที่คิดว่าดีที่สุดในขณะนั้น โดยไม่คำนึงถึงผลที่ตามมาในอนาคต นี่คือส่วนที่ “Greedy”
  4. ย้ายไปยังสถานะใหม่ตามตัวเลือกที่เลือก ซึ่งจะเป็นจุดเริ่มต้นใหม่ของการเลือกในรอบต่อไป
  5. ทำซ้ำขั้นตอน 2-4 จนกว่าจะถึงเป้าหมายหรือไม่สามารถดำเนินการต่อไปได้อีก

Greedy Algorithm (cont.)

- สิ่งที่ต้องพิจารณา
  - เป้าหมาย – ต้องการทำอะไรให้ดีที่สุด
  - ในแต่ละขั้นตอน มีตัวเลือกอะไรบ้าง เมื่อเลือกแล้วทำให้อะไรเปลี่ยนไปบ้าง
  - จะรู้ได้อย่างไรว่า ตัวเลือกไหนเป็นตัวเลือกที่ดีที่สุด
  - อาจต้องมีการเตรียมข้อมูลก่อน
  - จบการทำงานเมื่อไหร่

Example: Making Change (cont.)

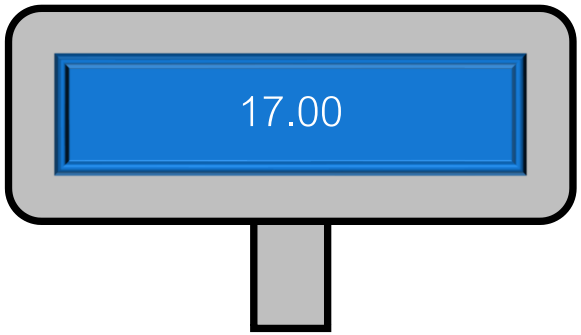
- ให้ c[i] เป็น array เก็บมูลค่าของเหรียญไว้
  - c[1] ที่มีมูลค่าน้อยที่สุด
  - c[k] มีมูลค่ามากที่สุด
- ให้จำนวนเหรียญที่ใช้เป็น a
- เป้าหมาย: ทอนเงิน n บาทโดยใช้**<u>จำนวนเหรียญน้อยที่สุด</u>**
- ตัวเลือกในแต่ละขั้น: เลือกเหรียญ c[i] มาทอนเงินที่เหลือ (j)
  - หักเงินที่เหลือออกตามจำนวนเหรียญ (j = j – c[i])
  - เพิ่มจำนวนเหรียญที่ใช้ 1 เหรียญ (a = a + 1)
- หยุดเมื่อ
  1. ทอนเงินได้ครบ หรือ
  2. เงินที่เหลือมีค่าน้อยกว่าเหรียญที่มีค่าน้อยที่สุด (ไม่สามารถทอนได้)

Greedy Algorithm for Making Change

- ตัวเลือกที่ดีที่สุด เป็นเหรียญมูลค่ามากที่สุดที่ยังสามารถใช้ได้ ($c[i] \leq j$)
- Greedy Algorithm: ใช้เหรียญที่มีค่ามากที่สุดจนไม่สามารถใช้ได้อีก แล้วจึงพิจารณาเหรียญที่มีค่าน้อยกว่าถัดไป จนกว่า:
  - ทอนเงินได้ครบ หรือ
  - เงินที่เหลือมีค่าน้อยกว่าเหรียญที่มีค่าน้อยที่สุด (ไม่สามารถทอนได้) → Return -1
- โค้ดนี้จะได้เฉพาะจำนวนเหรียญ หากต้องการทราบว่าใช้เหรียญอะไรบ้าง ต้องทำอย่างไร?

```
a = 0;
j = n;
i = k;
while j > 0
 if j >= c[i]
    a = a + 1
    j = j - c[i]
 else
    if i == 0
       return -1
    else
       i = i - 1
return a;
```

How it works



- จะต้องทอนเงิน 17 บาท
- มีเหรียญ 1, 2, 5, และ 10 บาท

1. a = 0, j = 17, i = 4, c[i] = 10 (j >= c[i])
2. a = 1, j = 7, i = 4, c[i] = 10 (j < c[i], i ≠ 0)
3. a = 1, j = 7, i = 3, c[i] = 5 (j >= c[i])
4. a = 2, j = 2, i = 3, c[i] = 5 (j < c[i], i ≠ 0)
5. a = 2, j = 2, i = 2, c[i] = 2 (j >= c[i])
6. a = 3, j = 0, i = 2, c[i] = 2

คำตอบ: 3 เหรียญ

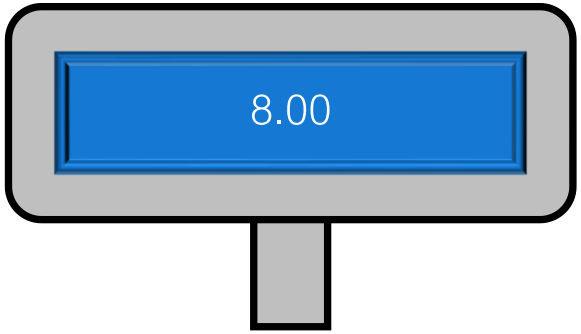
วิเคราะห์ Greedy Algorithm

- นับจำนวนการเปรียบเทียบใน while loop
- Loop แต่ละครั้ง จะใช้การเปรียบเทียบอย่างมาก 3 ครั้ง
- เราสามารถประมาณจำนวนครั้งที่เราจะ Loop ได้
  - อย่างมาก k ครั้ง จากการเปลี่ยนเหรียญ
  - อย่างมาก n ครั้ง จากใช้เหรียญ
- Time complexity: O(n + k)
- Space complexity: O(k)
- **<u>จะทำให้เร็วกว่านี้ได้ไหม?</u>**

```
a = 0;
j = n;
i = k;
while j > 0
 if j >= c[i]
    a = a + 1
    j = j - c[i]
 else
    if i == 0
       return -1
    else
       i = i - 1
return a;
```

ข้อจำกัดในการแก้ปัญหาทอนเงินด้วย Greedy Algorithm

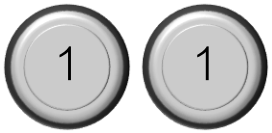
- จะต้องทอนเงิน 8 บาท
- มีเหรียญ 1, 4, และ 6 บาท



- คำตอบที่ได้จาก Greedy Algorithm คือ?
- จำนวนเหรียญที่น้อยที่สุดคือ?

ถ้าใช้ Greedy Algorithm

- จาก 8 บาท ใช้เหรียญ 6 บาท 1 เหรียญ

- เงินที่เหลือ 2 บาท สามารถทอนได้ด้วยเหรียญ 1 บาทสองเหรียญเท่านั้น 

- ใช้เหรียญทั้งหมด 3 เหรียญ

- เป็นจำนวนเหรียญเงินทอนที่น้อยที่สุดจริงหรือไม่?
- คิดว่า Greedy Algorithm พลาดอย่างไร?
- **<u>ถ้าตัวเลือกที่ดูดีที่สุดในแต่ละขั้นตอนไม่ใช่ตัวเลือกที่ทำให้ได้คำตอบที่ดีที่สุด Greedy Algorithm อาจให้คำตอบที่ไม่ดีที่สุด (Non-optimal) ได้</u>**

Another Example: Knapsack Problem

- จัดของใส่กระเป๋าอย่างไร ให้ได้คุณค่า มากที่สุด
- มีกระเป๋า 1 ใบ มีของ n ชิ้น
- Input: W น้ำหนักที่กระเป๋ารองรับได้

  w_i น้ำหนักของสัมภาระชิ้นที่ i

  v_i คุณค่าของสัมภาระชิ้นที่ i
- เป้าหมาย: เลือกสัมภาระ (x_i) ที่จะใส่กระเป๋าให้ได้**<u>มูลค่ารวมมากที่สุด</u>** โดยที่น้ำหนักรวมไม่เกินน้ำหนักที่ กระเป๋ารองรับได้

ทำให้ $\sum_{i=1}^{n} x_i v_i$ ได้ค่ามากที่สุด โดยที่ $\sum_{i=1}^{n} x_i w_i \leq W$

Knapsack (ต่อ)

- <u>ถ้า สามารถตัดแบ่งของแต่ละชิ้นได้</u>
- ตัวเลือก: เลือกของใส่กระเป๋า โดยสามารถเลือกเพียงบางส่วนของของ 1 ชิ้นได้ $0 \leq x_i \leq 1$
- สามารถหา Optimal Solution โดยใช้ Greedy Algorithm เลือกของที่มี<u>**อัตราส่วนมูลค่าต่อน้ำหนัก**</u>มากที่สุดตามลำดับ
- จนกระทั่งถึงของชิ้นที่ไม่สามารถใส่กระเป๋าได้ทั้งหมด ก็ใส่แค่ส่วนที่จะใส่ได้ และจะได้มูลค่าตามสัดส่วนนั้น

ปัญหา Knapsack (ต่อ)

- ตัวอย่าง
- W = 11

i	v_i	w_i	v_i / w_i
1	1	1	1.00
2	6	2	3.00
3	18	5	3.60
4	22	6	3.67
5	28	7	4.00

1. เรียงตาม v_i / w_i จากมากไป หา น้อย:
   ได้ชิ้นที่ 5, 4, 3, 2, 1.
2. เลือกชิ้นที่ 5 ทั้งชิ้น เหลือความจุ 4 หน่วย ได้มูลค่า 28 หน่วย
3. เลือกชิ้นที่ 4 ได้แค่ 4 หน่วยจาก 6 หน่วย ได้มูลค่า (4/6) × 22 = 14.67 หน่วย
4. รวมมูลค่าได้ 28 + 14.67 = 42.67 หน่วย

```
// item[] already sorted by v/w
i = 0;
totalV = 0;  // total value in knapsack
while(W > 0)
   if(W >= item[i].w)
         totalV += item[i].v
      W -= item[i].w
   else
      totalV += item[i].v * (W/item[i].w)
      W = 0
      i++
   }
return totalV
```

ข้อจำกัดของ Greedy Algorithm – 0/1 Knapsack

0/1 Knapsack

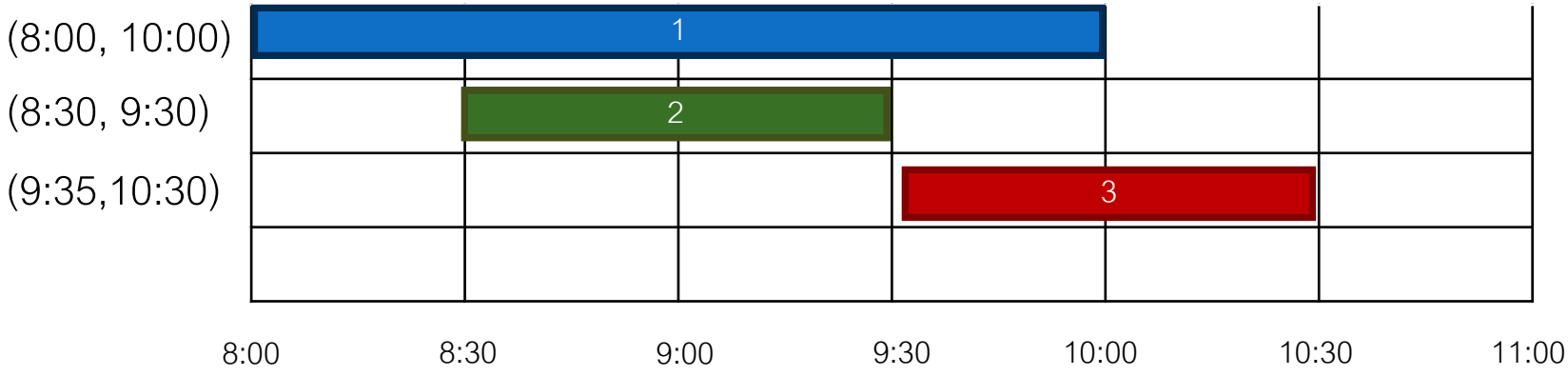
- แบ่งของไม่ได้ ถ้าจะเก็บใส่กระเป๋าต้องใส่ทั้งชิ้น
- ตัวเลือก: x_i เป็นได้แค่ 0 หรือ 1 เท่านั้น
- W = 11
  - Optimal solution คือ เลือกชิ้นที่ 3 และ 4 ได้มูลค่า 40 หน่วย
  - ถ้าเลือกชิ้นที่ 5 จะเลือกได้อีกแค่ชิ้นที่ 1 และ 2 ได้มูลค่าเพียง 35 หน่วย
- ทางเลือกตรงหน้าที่ดูดีที่สุด อาจไม่ใช่องค์ประกอบของ Optimal Solution

PROBLEMS

เนื้อหาจาก ผศ.ดร.จักริน ชวชาติ

C2P94 – แข่งกีฬาสี

- ต้องการให้นักกีฬาลงแข่ง<u>ในจำนวนรายการแข่งขันที่มากที่สุดที่เป็นไปได้</u>
  - แต่ละรายการมีเวลาเริ่ม (s_i) และเวลาจบ (t_i)
  - หากสองรายการมีเวลาคาบเกี่ยวกัน จะไม่สามารถลงแข่งทั้งสองรายการได้
  - จากข้อมูลรายการแข่งที่มีให้ จงหาว่าจะลงแข่งได้กี่รายการ

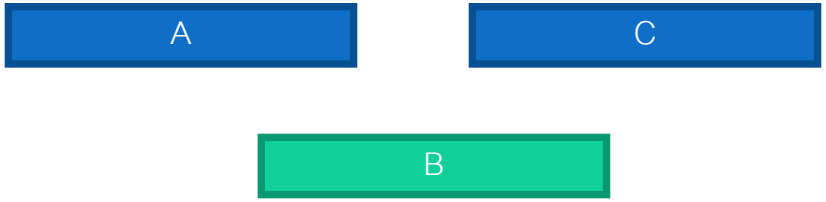


Interval scheduling

- ปัญหาแบบนี้ เรียกว่า Interval Scheduling กำหนดให้
  - **งาน/กิจกรรม**ที่ j มีเวลาเริ่มต้น **start** s_j และเวลาสิ้นสุด **stop** t_j

    เราอาจจะมองเป็น

  - งานสองงานจะ**สอดคล้องกัน** (Compatible หรือทำพร้อมกันได้ มันไม่**ซ้อน**ทับกัน)



A สอดคล้องกับ C

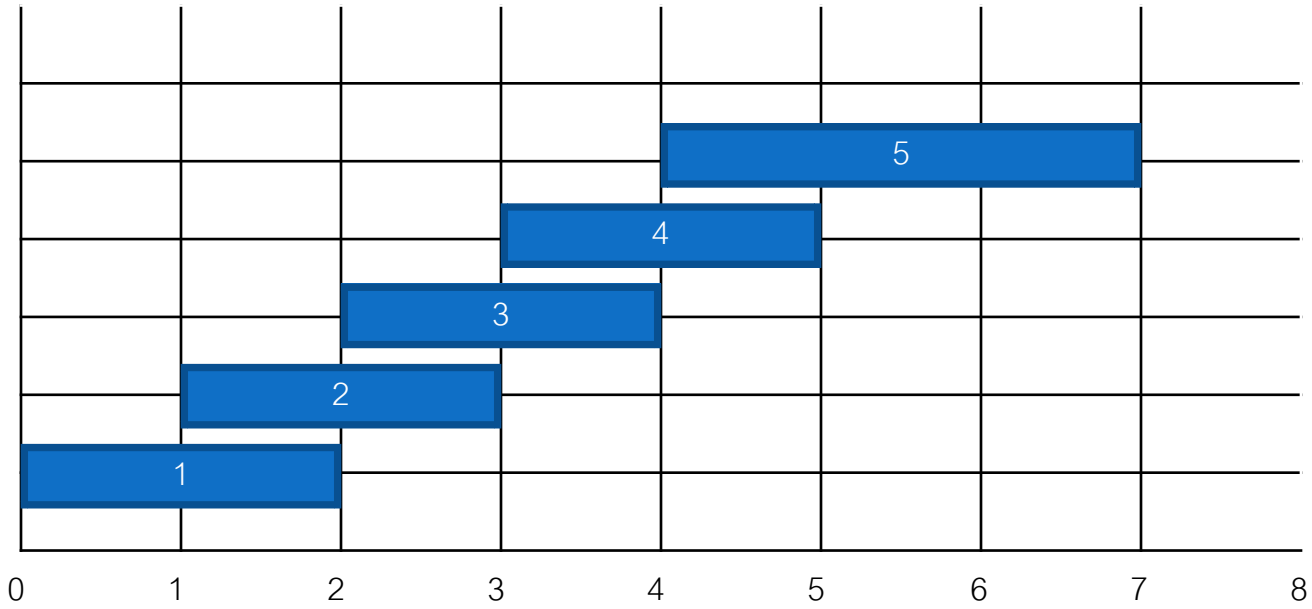
B ไม่สอดคล้องกับ A

B ไม่สอดคล้องกับ C

  - **เป้าหมายของปัญหานี้**คือ หาจำนวน subset ของงานที่สอดคล้องกันที่มีขนาดใหญ่ที่สุด จำนวนมากสุดนั่นเอง

ตัวอย่าง

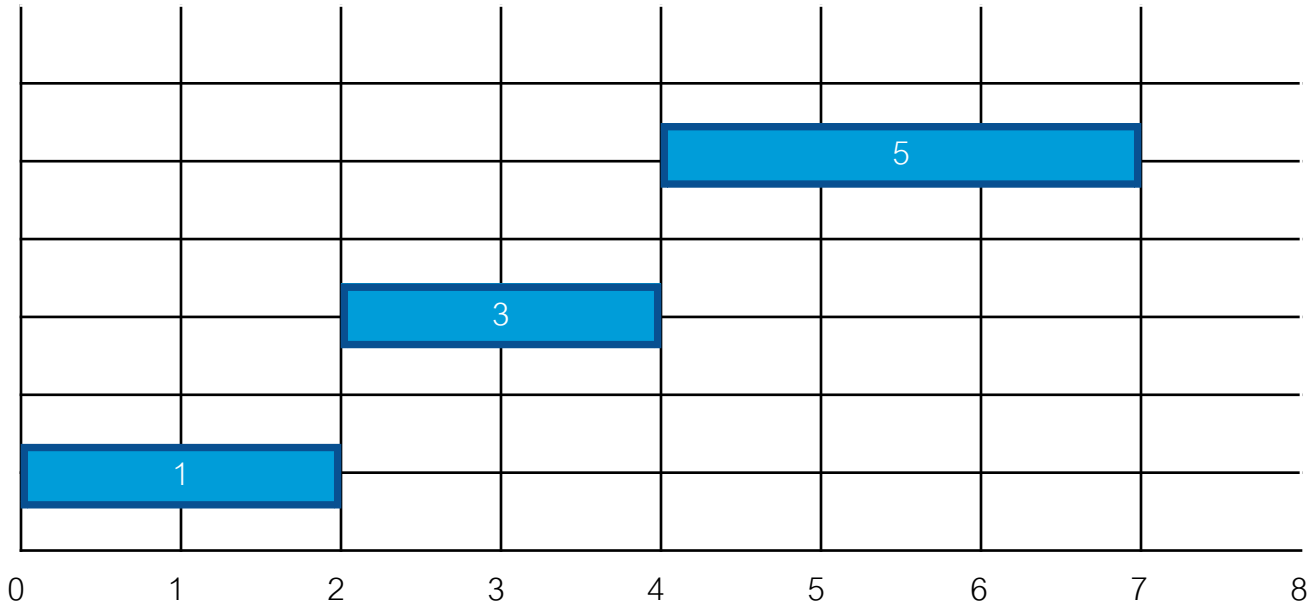
- มีงาน [0,2], [1,3], [2,4], [3,5], [4,7]



- ตัวอย่างนี้เลือกจำนวน subset ที่สอดคล้องกันที่ใหญ่ที่สุดได้งานใดบ้าง

ตัวอย่าง

- มีงาน [0,2], [1,3], [2,4], [3,5], [4,7]



- ตัวอย่างนี้เลือก [0,2], [2,4], [4,7] จำนวน 3 งาน

- ปัญหานี้อาจจะพบลักษณะการถามเช่น
  - มีงานที่มีช่วงเวลาทำงานชัดเจน ต้องการกำหนดให้เครื่องคอมที่ทำงานได้ทีละชิ้น ให้ได้จำนวนงานมากที่สุด
  - มีกิจกรรมที่มีช่วงเวลาจัดชัดเจน ต้องการเข้าร่วมกิจกรรมในหนึ่งวัน จำนวนมากที่สุด ไม่สามารถทำกิจกรรมสองอย่างพร้อมกันได้
  - มีห้องชมรม/ สนามกีฬา ที่มีช่วงเวลาการจัดชัดเจน ต้องการจัดแข่งกีฬาให้ได้จำนวนการแข่งมากที่สุด โดยกีฬาไม่สามารถแข่งสนามเดียวกันในเวลาเดียวกันได้
  - ... และอื่นๆ สังเกต แต่ละงาน มี**ช่วงเวลาจัด ห้ามซ้อนทับกัน**

แนวทางการแก้ปัญหา

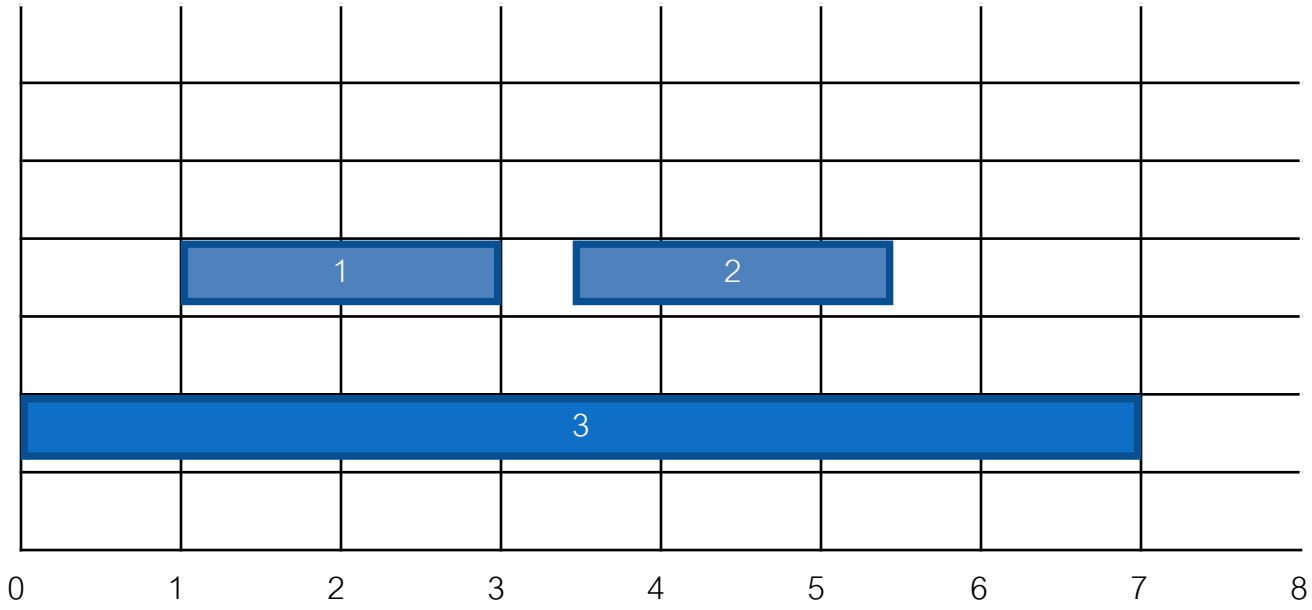
- เราอาจจะลองแนวทางในลักษณะนี้
  - แนวทางที่ 1 เลือกงานที่มีเวลาเริ่มต้นเร็วที่สุดก่อน
  - แนวทางที่ 2 เลือกงานที่มีเวลาสิ้นสุดเร็วที่สุดก่อน
  - แนวทางที่ 3 เลือกงานที่เวลาน้อยที่สุดก่อน
  - แนวทางที่ 4 เลือกงานที่เวลามากที่สุดก่อน
  - แนวทางที่ 5 เลือกงานที่ซ้อนทับงานอื่นน้อยที่สุดก่อน
  - แนวทางที่ 6 เลือกงานที่ซ้อนทับงานอื่นมากที่สุดก่อน

ข้อสังเกต เราเลือกสิ่งที่ดีที่สุดในขณะนั้นในการแก้ปัญหา

- ให้ลองยกตัวอย่างว่าแนวทางใดทำได้ หรือแนวทางใดมีข้อขัดแย้ง

แนวทางที่ 1 เลือกงานที่มีเวลาเริ่มต้นเร็วที่สุดก่อน

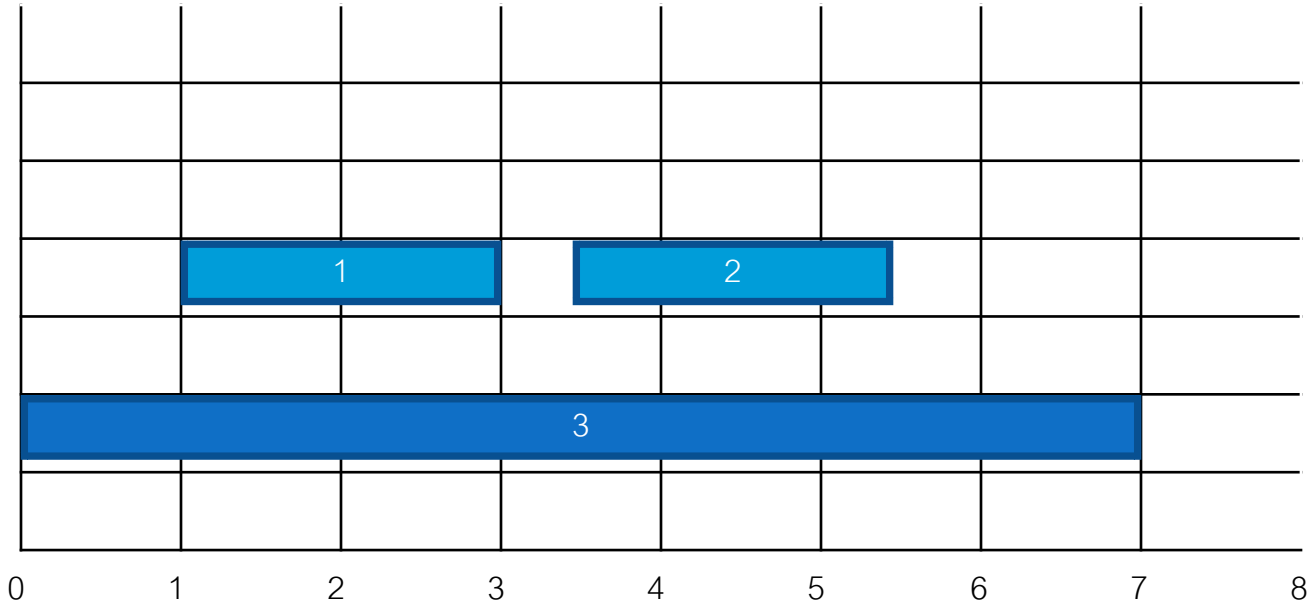
- แนวทางนี้คือการเลือกงานที่ s_j ของงานที่มีค่าน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง
- ถ้างานเป็นแบบนี้หละ



- ถ้าเลือกงาน 3 เราก็เลือกงาน 1 และ 2 ไม่ได้

แนวทางที่ 1 เลือกงานที่มีเวลาเริ่มต้นเร็วที่สุดก่อน

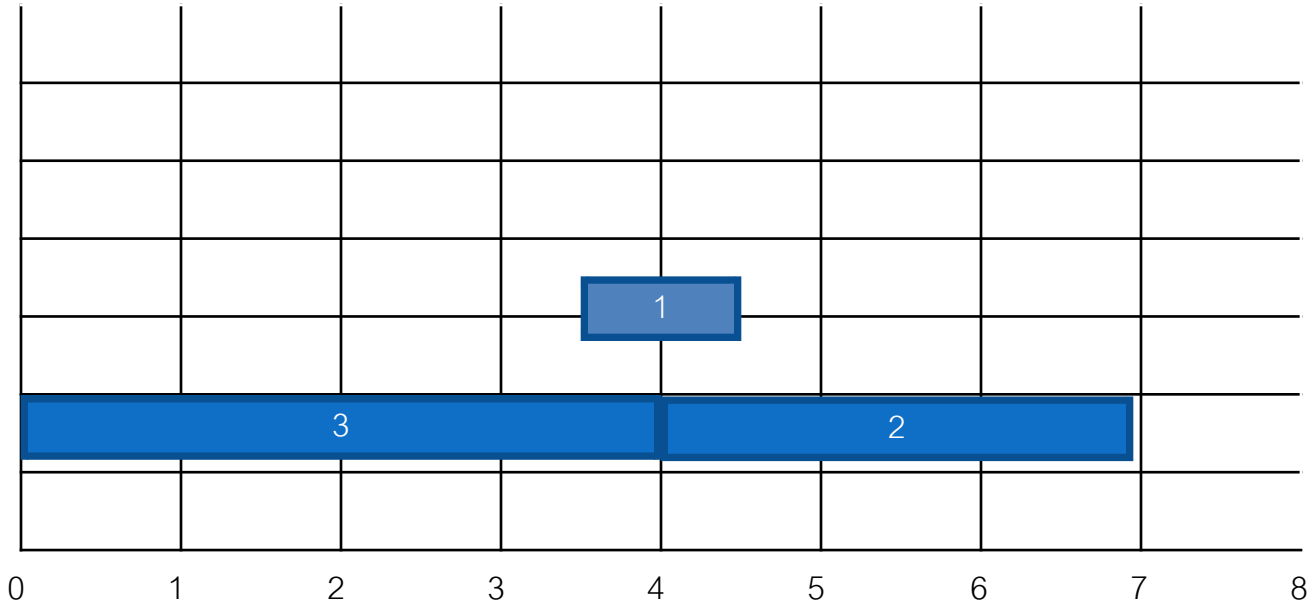
- แนวทางนี้คือการเลือกงานที่ s_j ของงานที่มีค่าน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง
- ถ้างานเป็นแบบนี้หละ



- คำตอบที่ดีที่สุดเป็น 1 และ 2

แนวทางที่ 3 เลือกงานที่เวลาน้อยที่สุดก่อน

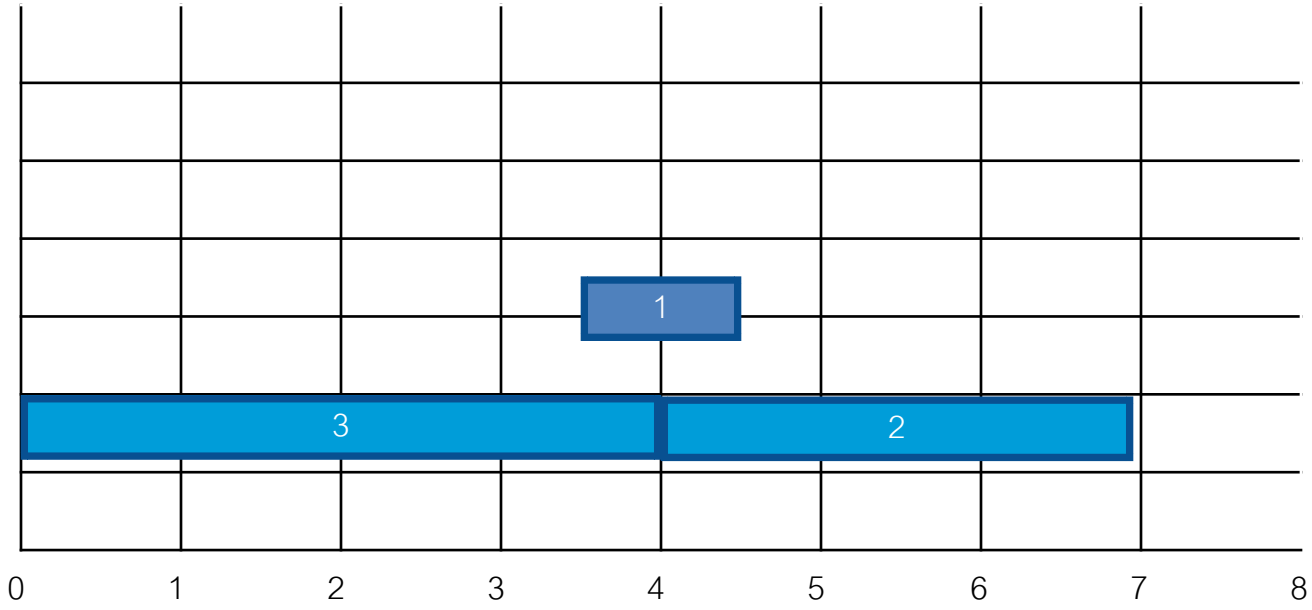
- แนวทางนี้คือการเลือกงานที่ $t_j - s_j$ ของงานที่มีค่าน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง
- ถ้างานเป็นแบบนี้หละ



- ถ้าเลือกงาน 1 เราก็เลือกงาน 3 และ 2 ไม่ได้

แนวทางที่ 3 เลือกงานที่เวลาน้อยที่สุดก่อน

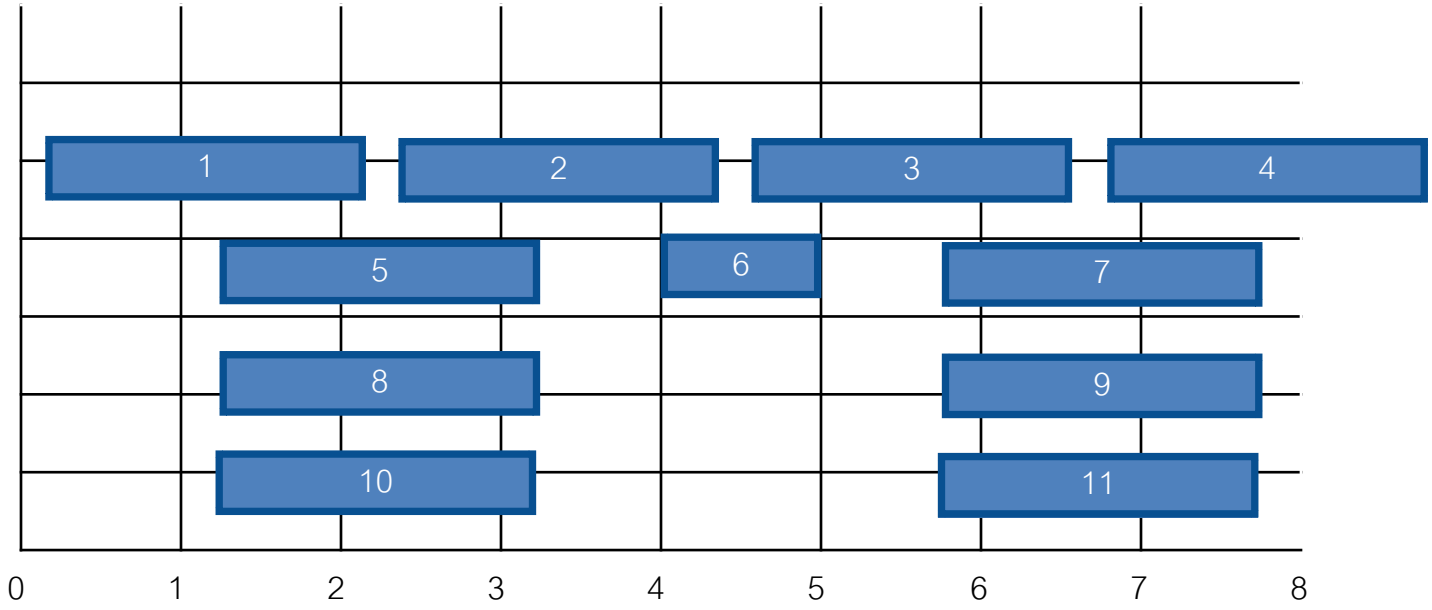
- แนวทางนี้คือการเลือกงานที่ $t_j - s_j$ ของงานที่มีค่าน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง
- ถ้างานเป็นแบบนี้หละ



- คำตอบที่ดีที่สุดเป็น 2 และ 3

แนวทางที่ 5 เลือกงานที่ซ้อนทับงานอื่นน้อยที่สุดก่อน

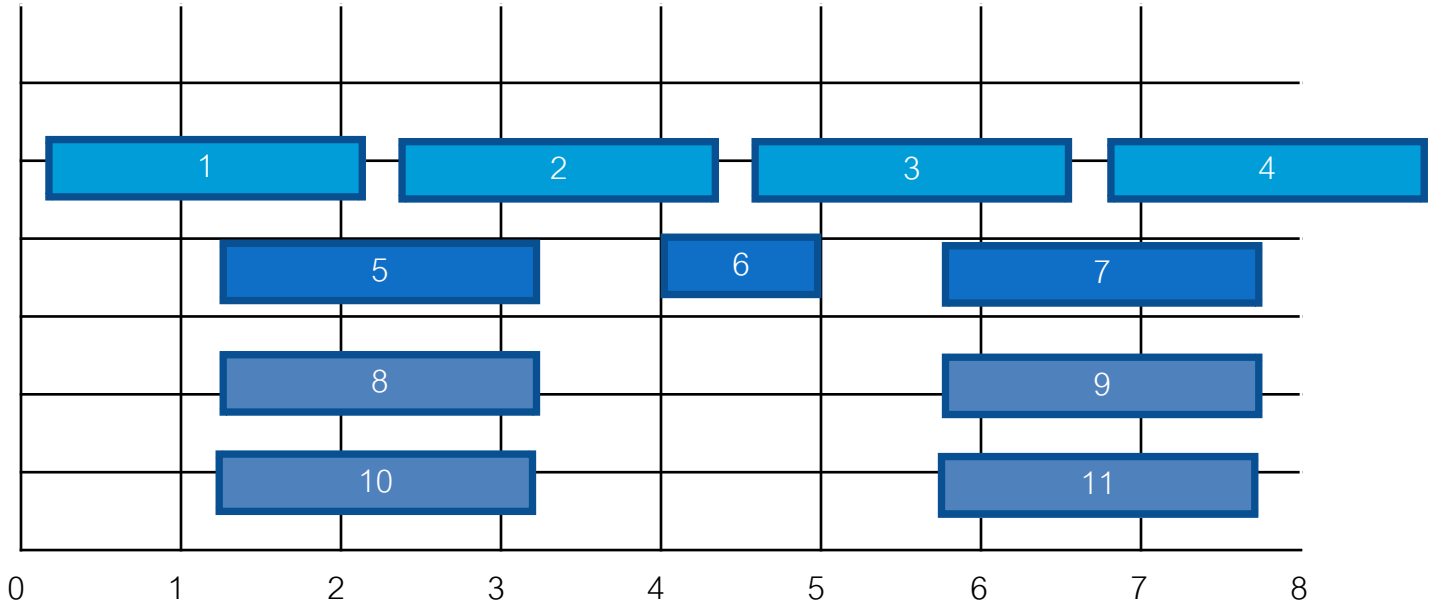
- แนวทางนี้คือการเลือกงานที่ซ้อนทับกับงานอื่นเป็นจำนวนน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง
- ถ้างานเป็นแบบนี้หละ



- ถ้าเลือกงาน 6 ก่อนผลจะเป็นแบบไหน

แนวทางที่ 5 เลือกงานที่ซ้อนทับงานอื่นน้อยที่สุดก่อน

- แนวทางนี้คือการเลือกงานที่ซ้อนทับกับงานอื่นเป็นจำนวนน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง
- ถ้างานเป็นแบบนี้หละ



คำตอบที่ดีที่สุดคือ 1, 2, 3 และ 4

แนวทางที่ 2 เลือกงานที่มีเวลาสิ้นสุดก่อน

- แนวทางนี้คือการเลือก stop ของงานที่ j หรือ t_j ที่มีค่าน้อยที่สุดมาเป็นคำตอบก่อนนั่นเอง

- แนวคิดนี้เปรียบได้กับ

ถ้าเราเลือกงานที่เสร็จเร็วก่อน

ดังนั้นจะมีเวลาเหลือให้เราเลือกงานอื่นๆ มากขึ้นนั่นเอง

สำหรับข้อนี้

- Greedy algorithm สำหรับปัญหานี้คือ
  - เราจะพิจารณางานตามเวลาเสร็จเร็วที่สุดก่อน
  - เลือกงานที่สอดคล้องกับงานก่อนหน้าที่เลือกไปแล้วรวมเป็นคำตอบ

Sort jobs by finish time -> $f_1 \leq f_2 \leq f_3 \ldots \leq f_n$

```
A = {}
for j =1 to n {
    if(job j compatible with A)
        A = A U {j}
}
```

ปัญหาที่ต้องจัดการ

- เราจะเก็บข้อมูลเวลา และเรียงตามเวลาเสร็จได้อย่างไร
- เราต้องเก็บข้อมูลสองอย่าง start กับ stop

```
struct Interval {
    int start, stop, index;
    bool operator<(const Interval& i)const {
        if (stop != i.stop) return stop < i.stop;
        else return start < i.start;
    }
};
struct Interval s[n];
sort(s, s+n);
```

หรือจะใช้ vector of pair
แล้วเก็บข้อมูลตัวแรกของ pair เป็นเวลาเสร็จ ข้อมูลตัวที่สองเป็นเวลาเริ่มก็ได้ แล้วใส่ใน Priority queue

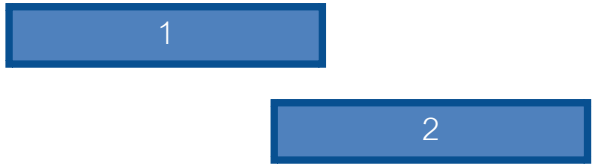
```
vector<pair<int,int>>ans;

priority_queue<pair<int,int>,vector<pair<int,int>>,greater<pair<int,int>>>Activities;
int n, start, stop;


cin>>n;
for(int i=0;i<n;i++){
   cin>>start>>stop;
   Activities.push(make_pair(stop,start));
}
```

ปัญหาที่ต้องจัดการ

- Compatible ตรวจสอบยังไง
- งาน 2 งานสอดคล้องกัน

  - เวลาเริ่มงานงานที่สอง มากกว่าหรือเท่ากับเวลาจบของงานแรก

- งาน 2 งานไม่สอดคล้องกัน



  - เวลาเริ่มงานงานที่สอง น้อยกว่าเวลาจบของงานแรก

❑การตรวจสอบความสอดคล้องทำได้ไม่ยาก

เมื่อเราเรียง งานตามเวลาเลิกแล้ว (sort(s,s+n);)

```
int endt=0; //หรือจะเอาตัวแรกมาก่อนก็ได้แล้วแต่ขอบเขต
for(int j=0; j<n; j++){
    if(s[j].start>=endt){
        answer.push_back(s[j].index);
        endt=s[j].stop;
    }
}
```

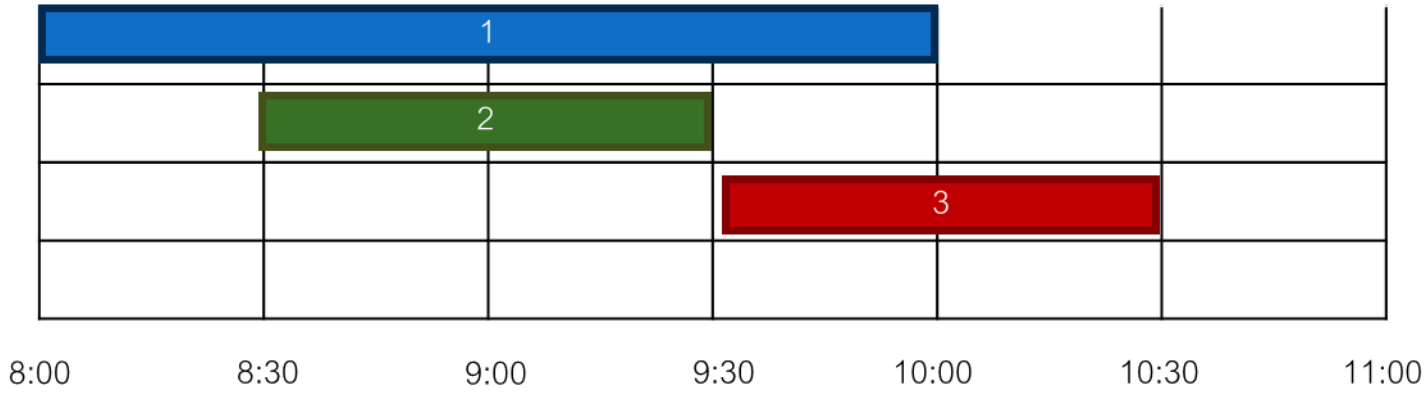
```
auto it = Activities.top();
start = it.second;
stop = it.first;
Activities.pop();
ans.push_back(make_pair(start,stop));
while(!Activities.empty()){
    it = Activities.top();
    if(it.second >= stop){
        start = it.second;
        stop = it.first;
        ans.push_back(make_pair(start,stop));
    }
    Activities.pop();
}
```

- ทีนี้ก็แสดงผล ขึ้นกับโจทย์แล้วว่าจะให้แสดงผลเป็น
  - ลำดับของงาน
  - จำนวนของงาน
  - งานแต่ละอันที่แสดงเวลาเริ่ม เวลาเสร็จ

```
for(auto itr=ans.begin();itr!=ans.end();itr++){
        cout << (*itr).first << " " << (*itr).second <<
endl;
}
```

C2P95 – จัดการการแข่งขันกีฬา

- มีข้อมูลรายการแข่งขันคล้ายกับข้อ C2P94 แต่ต้องการจัดรายการแข่งขันโดยที่ใช้จำนวนสนามน้อยที่สุด
  - แต่ละรายการมีเวลาเริ่ม (s_i) และเวลาจบ (t_i)
  - สนามหนึ่ง สามารถใช้จัดการแข่งขันอะไรก็ได้
  - แต่หากสองรายการมีเวลาคาบเกี่ยวกัน จะไม่สามารถใช้สนามเดียวกันได้ ต้องเปิดสนามใหม่



Interval Partitioning

ปัญหาแบบนี้เรียกว่า Interval partitioning คล้ายกับ Interval Scheduling ที่มีงาน โดยงานแต่ละงานมีเวลาเริ่ม start และเวลาเลิก stop มาให้

งานแต่ละงานจะ**สอดคล้องกัน** (Compatible) ถ้าเวลาของงานไม่ล้ำเหลื่อม (Overlap) กัน

- **สิ่งที่แตกต่าง**คือ เราต้องการแบ่งงานออกเป็น k กลุ่มโดยที่แต่ละกลุ่มของงานสอดคล้องกันหมด และ k มีค่าน้อยที่สุด - ทุกงานต้องอยู่ในกลุ่มใดกลุ่มหนึ่ง

ตัวอย่างเช่น

- ในภาควิชามีหลายวิชาที่ต้องการเปิดสอนในวันวันหนึ่ง แต่ละวิชามีเวลาเริ่มเรียนและเลิกเรียน ภาควิชาจะต้อง**เปิดห้องเรียนน้อยที่สุด**กี่ห้องเพื่อให้ทุกวิชาสามารถมีการเรียนการสอนได้ ทั้งนี้แต่ละวิชา เรียนเวลาเหลื่อมกันไม่ได้

Interval Partitioning

- วิชาที่ j มีเวลาเริ่ม s_j เวลาเลิก t_j
- **เป้าหมาย** ต้องการหาจำนวนห้องเรียนที่น้อยที่สุดที่จัดการเรียนการสอนได้โดยที่ไม่มีสองวิชาใด ๆ การเรียนการสอนในเวลาเดียวกัน ในห้องเดียวกัน

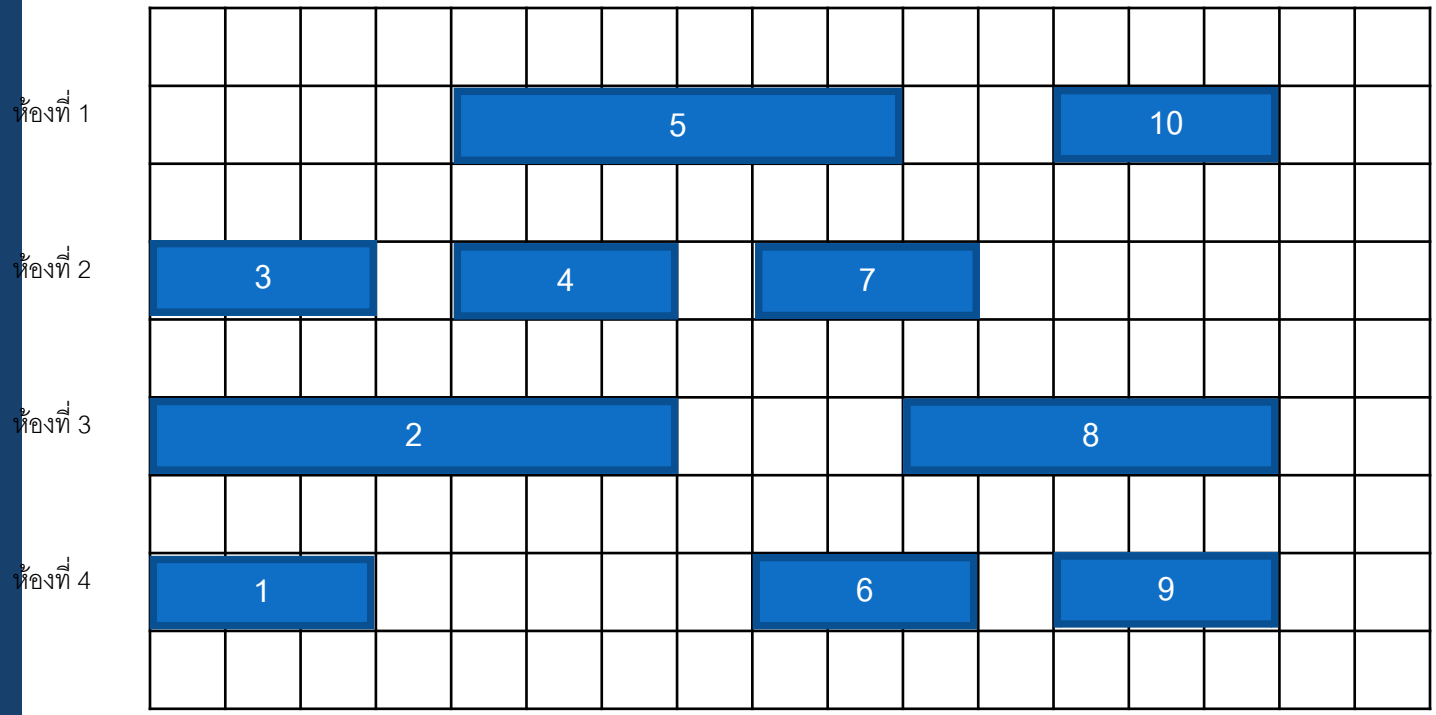
- ลองดูตัวอย่างต่อไปนี้

  กำหนดให้ (start, stop) ของแต่ละวิชาให้ จะต้องเปิดห้องเรียนกี่ห้อง

  (0,3), (0,7), (0,3), (4,7), (4,10), (8,11), (8,11), (10,15), (12,15), (12,15)

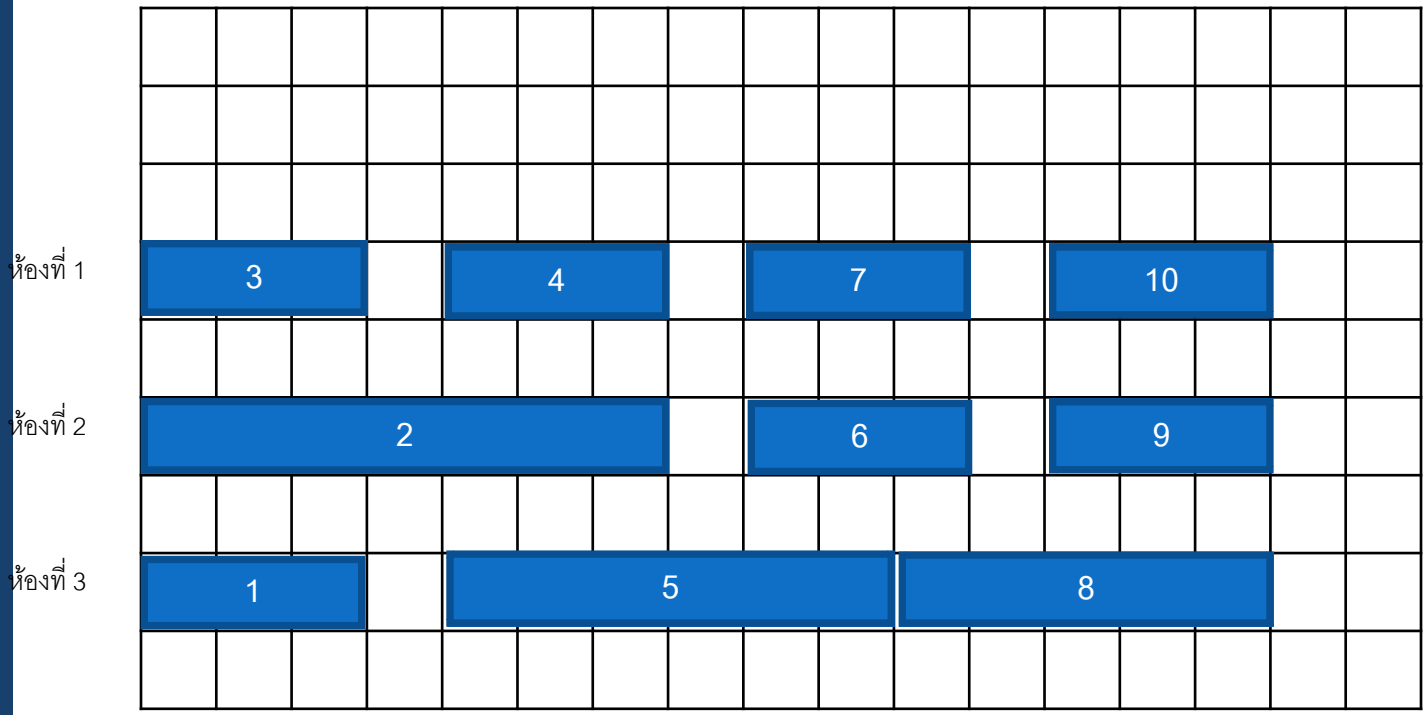
Interval Partitioning

- อาจจะจัดวิชาได้ 4 ห้องดังรูป



Interval Partitioning

- แต่จริง ๆ ใช้เพียง 3 ห้องดังรูป



แนวทางการแก้ปัญหา

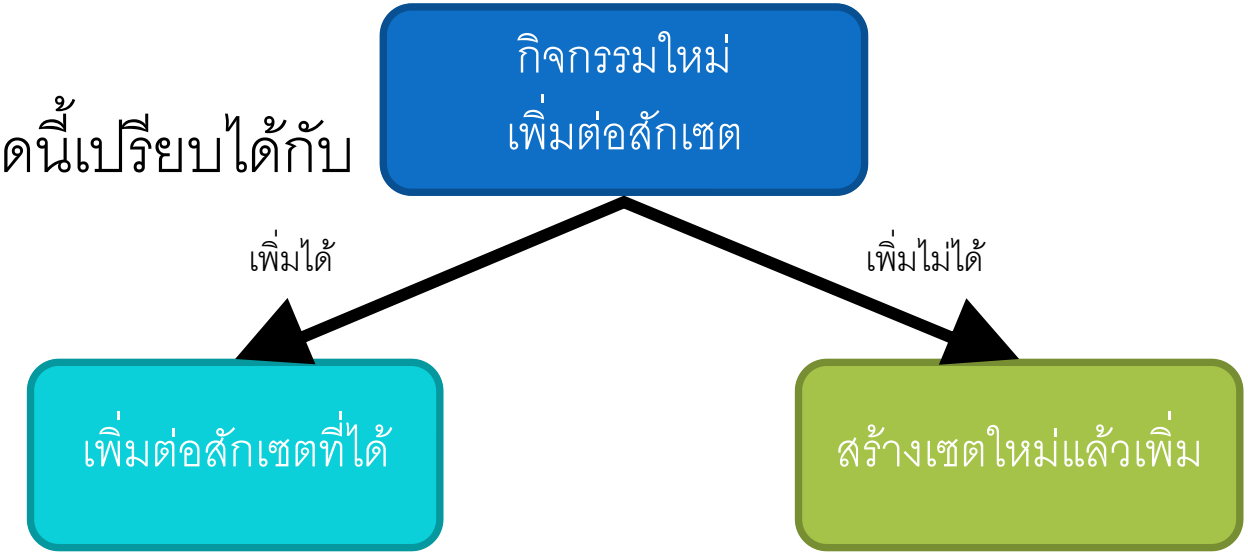
- เราอาจจะลองแนวทางในลักษณะนี้
  - แนวทางที่ 1 เลือกงานที่มีเวลาเริ่มต้นเร็วที่สุดก่อน
  - แนวทางที่ 2 เลือกงานที่มีเวลาสิ้นสุดเร็วที่สุดก่อน
  - แนวทางที่ 3 เลือกงานที่เวลาน้อยที่สุดก่อน
  - แนวทางที่ 4 เลือกงานที่เวลามากที่สุดก่อน
  - แนวทางที่ 5 เลือกงานที่ซ้อนทับงานอื่นน้อยที่สุดก่อน
  - แนวทางที่ 6 เลือกงานที่ซ้อนทับงานอื่นมากที่สุดก่อน
  - ...

เลือกแบบไหนดี ลองคิดดู

แนวทาง เลือกงานที่มีเวลาเริ่มต้นก่อน

- แนวทางนี้คือการเลือก start ของงานที่ j หรือ s_j ที่มีค่าน้อยที่สุดมาพิจารณาก่อนนั่นเอง

- แนวคิดนี้เปรียบได้กับ



- คำถาม จำนวนเซตจะมากไหม

  ไม่มากเนื่องจากเราจะสร้างเซตใหม่เมื่อจำเป็นเท่านั้นเอง

สำหรับข้อนี้

- Greedy algorithm สำหรับปัญหานี้คือ
  - เราจะพิจารณางานตามเวลาเริ่มต้นเร็วที่สุดก่อน
  - ถ้างานใหม่สอดคล้องกับงานก่อนหน้าเพิ่ม แต่ถ้าไม่ก็สร้างกลุ่มใหม่แล้วเพิ่ม

```
Sort jobs by start time -> s1 ≤ s2 ≤ s3 … ≤ sn

d = 0
for j =1 to n {
    if(job j compatible with some set k)
        Schedule job j in set k
    else
        Allocate a new set d+1
        Schedule job j in set d+1
        d = d+1
}
```

❑ compatible

- งานจะสอดคล้องกับกลุ่มหมายความว่า ไม่มีงานอื่นใช้เวลาเดียวกันในกลุ่มนั้น นั่นคือกลุ่มนั้นว่าง ในช่วงเวลาดังกล่าว
- ถ้ามองเป็นห้องเรียนก็คือ ห้องเรียนนั้นว่างสำหรับงาน/วิชา โดยที่ไม่ซ้อนทับกับงาน/วิชาอื่นนั่นเอง

❑ some

- หมายถึง มีเซตที่ compatible สักเซตหนึ่ง
- จุดนี้ทำให้สามารถมีคำตอบที่ดีที่สุดได้หลายคำตอบ

ในการจัดการปัญหา compatible

- เนื่องจากว่าเราต้อง sort
- เราจะเก็บงานไว้ใน priority queue ที่เรียงลำดับเวลาเริ่มจากน้อยไปมาก – หรือ sort ก็ได้

```
priority_queue<pair<int,int>,vector<pair<int,int>>,
greater<pair<int,int>>>job;
```

สมมติว่าจะเอาคำตอบแค่**จำนวนห้องที่ต้องใช้น้อยที่สุด**

- คำตอบ เราจะเก็บไว้ใน priority queue เช่นกัน โดยที่ของที่อยู่ใน priority queue จะเป็น job สุดท้ายของเซต/ห้อง นั่นคือ ถ้า priority queue นี้มีกี่อัน ก็คือจำนวนเซต/ห้องนั่นเอง

```
priority_queue<pair<int,int>,vector<pair<int,int>>,
greater<pair<int,int>>>ans;
```

แต่ในการเก็บ เราจะเอาเวลาเสร็จเป็นตัวแรกของ pair เพราะเราจะใช้ประโยชน์ที่ว่า ถ้ามีสักเซต เราจะเลือกเซตที่น้อยสุด ซึ่งตัวน้อยสุดในที่นี้คือ เวลาเสร็จเร็วสุด ถ้าตัวเร็วสุดยังเพิ่มไม่ได้แสดงว่าอะไร

ตัวอย่าง

- (0,3), (0,7), (0,3), (4,7), (4,10), (8,11), (8,11), (10,15), (12,15), (12,15)

(0,3) (4,7) (8,11) (12,15)

เซตที่ 1

(0,7) (8,11) (12,15)

เซตที่ 2

(0,3) (4,10) (10,15)

เซตที่ 3

❑ส่วนรับข้อมูล

```
priority_queue<pair<int,int>,vector<pair<int,int
>>,greater<pair<int,int>>>job, ans;
int n, start, stop;
cin>>n;
for(int i=0;i<n;i++){
   cin>>start>>stop;
   job.push(make_pair(start,stop));
}
```

```
auto it = job.top();
ans.push(make_pair(it.second,it.first));
job.pop();
int d=1;
for(int i=1;i<n;i++){
     it = job.top();
     auto it2 = ans.top();
     if(it2.first<=it.first){
          ans.pop();
          ans.push(make_pair(it.second,it.first));
     }else{
          ans.push(make_pair(it.second,it.first));
          d=d+1;
     }
     job.pop();
}
cout<<d;
```